

Week 6 — DevOps, Deployment & Observability: Report

Eungchan Kang · SWENG 861 Software Construction · Repository `psu-edu/sweng861-crud-ekk5635`, folder `week-06-devops/` · main at `33f401d` (2026-10-03) · CI runs on the mirror `yeschan119/sweng861-crud-ekk5635` · screenshots in `week-06-devops/docs/screenshots/`

1. CI/CD pipeline and container images

`.github/workflows/ci.yml` runs on every push and pull request. GitHub Actions is disabled on the `psu-edu` repository by the organization (enabling it returned 409), so the same commits are pushed to a public personal mirror and the runs live there. Issues, pull requests and merges stay on `psu-edu`.

Job	What it does	Fails the run when
backend	<code>ruff check .</code> , then <code>pytest --cov=.</code> against a PostgreSQL 17 service container	a lint rule, a test, or coverage under 80% (threshold in <code>.coveragerc</code>)
frontend	<code>npm ci --ignore-scripts</code> , lint, <code>test:coverage</code> , build	a lint rule, a test, coverage under 80% (<code>vitest.config.ts</code>), a type error
audit	<code>pip-audit</code> , <code>npm audit</code> , tracked- <code>.env</code> check (section 4)	a known vulnerable dependency or a committed <code>.env</code>
package	after the three above: builds both images tagged <code><commit SHA></code> and <code>week6</code> , smoke-tests them, checks non-root and no Node in the frontend image, scans both with Trivy	the build, the start, a root user, or a scan finding
alert-rules	<code>promtool check config</code> and <code>promtool test rules</code>	a rule error, a failing rule test, or no rule file loaded

Both images are multi-stage. The backend runs as uid 10001 on `python:3.14-slim` with no `pip`; the frontend serves only the built `dist/` from `nginx-unprivileged:1.30-alpine` as uid 101. The workflow token is read-only and images are not pushed to a registry. Evidence: a green run (01, 20), a run failed by a broken assertion (02–03), one failed by the coverage gate raised to 99% (04–05), and two security failures (16–19). The pipeline's use is documented in the README section *Continuous integration*.

2. Observability

Health and logs

`GET /health` and `/health/ready` return `{"status":"UP","db":"UP"}` (200) or `DOWN` (503) after a database check with a 2-second connect timeout on an unpooled connection; `/health/live` checks only the process. Every request writes one JSON line with `timestamp`, `level`, `method`, `path` (query string excluded), `status`, `duration_ms` and a `request_id` taken from `X-Request-ID` or generated, and returned in the same header. Login success and failure, coverage creation and unexpected errors are logged as structured events under the same ID; an unexpected error returns only an incident ID to the caller and logs the traceback next to it. A test asserts that no log line carries a token, cookie, OAuth code, configured secret or email. Screenshot 06 follows three requests by their IDs; screenshot 21 shows `/health` going from UP to DOWN when the database stops, and the `unexpected_error` line for the resulting 500, matched to the response by incident ID.

Metrics, SLOs and dashboard

The API exports `http_requests_total{method,route,status}` and `http_request_duration_seconds{method,route}`, labelled by route template so that IDs in paths cannot create unbounded series, plus two domain counters, `coverages_created_total` and `logins_total{outcome}`. Prometheus scrapes `/metrics` inside the compose network every 15 s; the gateway refuses that path from outside.

SLI (7-day window)	SLO	How it is measured
--------------------	-----	--------------------

Login success: successful logins over logins that failed for a reason the service owns	$\geq 99\%$	<code>logins_total</code> by outcome plus 5xx on <code>/auth/callback</code> ; a declined consent or forged state is the caller, not the service
API availability: <code>/api/*</code> requests not ending in 5xx	$\geq 99.5\%$	<code>http_requests_total</code> by status class
Latency: p95 of GET <code>/api/coverages</code>	≤ 500 ms	<code>histogram_quantile(0.95, ...)</code> over the duration histogram (0.5 s is a bucket edge)

Dashboard (screenshot 07). Grafana dashboard *SWENG 861 · API*, provisioned from files, on data from the running stack. Top row: the three SLOs over the selected range. Below: request rate by route, error rate 4xx vs 5xx, p95 latency by route, coverages created, logins by outcome.

Using it in an incident. The top row says which promise is broken. A 5xx rise means the service is failing, while a 4xx rise alone usually means callers or a scan. Latency that climbs before the 5xx points at a slow dependency, the database or SEC EDGAR. A request rate that drops to zero with no errors points at the gateway or network. A route and a time from the panels lead to the logs, where one request is followed by its `request_id`. Screenshot 11 shows the dashboard during the drill in section 5.

3. Scalability and FinOps

3.1 Workload assumptions

The users are equity analysts, so traffic follows the trading day. Assumed: 1,000 monthly active users, 50 concurrent at peak; a login peak of 200 per minute at market open; 100 coverages created per hour; 300 EDGAR collections per hour at peak, each fetching several XBRL concepts for one company. Reads (list and detail) are about ten times the writes.

3.2 Bottlenecks

- The login rate limit sees one client.** `ratelimit.py` allows 10 logins per 60 s per key, and the key is `request.client.host`. Behind the gateway, uvicorn does not trust the gateway as a proxy, so every user has the gateway's address and all of them share one bucket; a 429 on login was observed during this week's testing. At the assumed 200 logins per minute, 10 succeed and about 95% are refused. This is the first thing to fail: it caps the whole service at 5% of the peak login rate. It is also per process: with more workers the effective limit multiplies.
- EDGAR collection runs inside the request.** A collection calls SEC synchronously, with up to 3 attempts inside a 15 s budget (`edgar.py`), on one of the threads of a single uvicorn process (the default thread pool is 40). At 300 collections per hour that is light on average, but at market open a burst of 40 slow collections fills the pool and every other request, including GET `/api/coverages`, queues behind them. The p95 SLO breaks before any error appears. SEC's fair-access policy also caps a client at 10 requests per second, which bounds a burst no matter how many workers run.
- The database pool is the default.** SQLAlchemy's default pool (5 connections plus 10 overflow) is created once per process. While collections hold threads and connections, the pool runs out before the thread pool does, and adding uvicorn workers multiplies connections against PostgreSQL's default limit of 100.

Component	Limit today (source)	Demand at assumed peak	Breaks when	First symptom
Login rate limit	10 per 60 s for one shared key (<code>ratelimit.py</code>)	200 per minute	the 11th login in any minute	429 on <code>/auth/login</code> ; login SLO looks healthy because 429 is a client error, so it hides in the 4xx panel
Request threads	40 threads, 1 process (AnyIO's default for sync endpoints)	bursts of collections at open, each up to 15 s	about 40 collections in flight	p95 of every route rises together; then gateway 504 after 30 s

DB connections	15 per process (5 + 10)	one per in-flight request	16 concurrent requests touching the DB	QueuePool limit timeouts, logged as unexpected_error, 5xx SLO burns
SEC EDGAR	10 requests per second per client (SEC policy)	several concepts per collection	about 3 collections per second	429 from SEC, retried, then the collection reports an upstream failure

The first row is the cheapest to fix and the most damaging, which sets the order of the changes below.

3.3 Improvements

Change	Performance	Cost
Start uvicorn with <code>--proxy-headers --forwarded-allow-ips=<gateway></code> so the limiter keys on the real client from <code>X-Forwarded-For</code> , and move the counters to Redis so the limit is exact across workers	Removes the 95% refusal at peak; one shared limit regardless of replica count	A small managed Redis (on the order of \$15–25/month); the flag itself is free
Cache EDGAR results by CIK and concept with a TTL of a day (filings change quarterly), and move collection to a queue with a worker; the API returns 202 and the SPA polls or is notified	Request threads are never held for 15 s; repeat collections become a cache hit; the worker can pace calls under SEC's 10 req/s	One extra worker process and the same Redis as the queue; fewer outbound calls
Size the pool to the workers: for example 4 workers × (pool 5 + overflow 5) = 40 connections, below PostgreSQL's limit with room for migrations; add PgBouncer if replicas grow	Predictable connection use; no exhaustion under a burst	None at this size; PgBouncer is one small container
Index <code>coverages (owner_id, created_at DESC)</code> , which matches the list query's filter and order	The list stays an index scan as rows grow	Negligible storage
Autoscale API replicas on p95 latency and request rate, not CPU	Capacity follows the market-open peak	Pay for peak hours only; scale to a minimum of 1 overnight

How each change would be verified. A load test (k6 or Locust) against the compose stack replays the assumed peak: a ramp to 200 logins per minute from distinct forwarded addresses, 50 users reading lists and details, and a burst of 40 collections against a stubbed EDGAR that answers in 10 s. A change passes when the three SLOs hold on the dashboard for the run and the 4xx panel shows no 429 on login. The same test run before the change gives the baseline, so each improvement is measured, not assumed.

3.4 FinOps

Monitoring cost. Every cloud resource would carry tags for `service` (api, frontend, db, observability) and `env`, and a cost dashboard would group spend by those tags with a monthly budget alert per environment. The metrics already exported give a cost-per-unit view: requests served and coverages created per day against the API's daily cost.

A rough monthly bill. The figures below are approximate list prices for a minimal AWS deployment in one region, to show where the money goes rather than to quote; they would be checked in the AWS pricing calculator before any deployment.

Resource	Size	Approx. per month	Note
API containers (ECS Fargate)	2 tasks × 0.5 vCPU / 1 GB, scaling to 4 at peak	\$35–60	largest variable cost; follows the autoscaling policy
Frontend	static <code>dist/</code> on S3 + CloudFront	under \$5	no container needed in production

PostgreSQL (RDS)	db.t4g.small, 20 GB, single AZ	\$25–35	runs all month, so the best candidate for a reservation
Redis (ElastiCache)	cache.t4g.micro	\$12–15	rate-limit counters, EDGAR cache, work queue
Load balancer	one ALB	\$18–25	replaces the compose gateway
Logs and metrics	CloudWatch logs, 14-day retention; managed Prometheus/Grafana or one small task	\$10–30	driven by log volume: one JSON line per request

The total is on the order of \$100–170 per month. Compute and the always-on database are about two thirds of it, so rightsizing and reservations matter more than anything else; log volume is the item most likely to grow unnoticed, because every request writes a line.

Optimization. Rightsize the API from the p95 latency and CPU at peak rather than a guess; run the development and demo stacks only during working hours, since the compose stack here idles all night; use a reserved or committed instance for the database, which runs constantly, and spot capacity only for the stateless EDGAR worker, whose jobs can be retried; drop waste such as repeated EDGAR calls for the same company (the cache above), scrape intervals shorter than the 15 s that the SLOs need, and log retention longer than the incident window, since Prometheus already keeps 8 days for the 7-day SLOs.

4. DevSecOps

4.1 Docker and secrets

Both runtime images run as non-root (checked in CI), carry no build tools, and contain no secret: `.env` is git-ignored and listed in `.dockerignore`, so it is never in the build context, and Trivy's secret scanner checks both images on every run. The secrets are `GOOGLE_CLIENT_SECRET`, `SESSION_JWT_SECRET` and `POSTGRES_PASSWORD`.

Environment	Where secrets live	How the app receives them
Local	<code>week-06-devops/backend/.env</code> , created from <code>.env.example</code> (placeholders only), never committed	<code>env_file</code> for the API and migration services, and <code>\${VAR}</code> interpolation from the same file for the database container and <code>DATABASE_URL</code> , both in <code>docker-compose.yml</code> at container start
CI	None needed today: the test database uses throwaway credentials for a container that exists only for the job. A real credential, such as a registry token, would be a GitHub Actions secret	<code>{{ secrets.NAME }}</code> in the step that needs it
Production (not deployed)	A cloud secret manager, such as AWS Secrets Manager	Injected as environment variables by the container platform; the image is the same in every environment

4.2 Security checks in CI

`pip-audit` fails on any known vulnerability in the backend requirements; `npm audit --omit=dev --audit-level=high` fails on high or critical advisories in runtime dependencies; Trivy 0.75.0 fails on a fixable HIGH or CRITICAL vulnerability, or a secret, in either image. Two choices are deliberate and documented in the README: dev dependencies are audited report-only, because `braces` (no fixed version yet) reaches only the lint configuration and the frontend image holds no Node code; and Trivy skips findings with no published fix, which a rebuild cannot act on. The first runs failed on real findings: `PyJWT 2.13.0` (fixed by 2.15.1), the base image's `pip` with vendored packages, `libpcre2`, and 42 Alpine packages under a stale `nginx-unprivileged:1.29-alpine` tag (screenshots 16–19); the fixed run is screenshot 20.

4.3 AI-assisted infrastructure review

An AI assistant drafted the Dockerfiles, CI jobs, health and logging code, Prometheus and Grafana configuration, alert rules and runbooks this week. Each first output was saved unchanged before review in `week-06-`

devops/docs/ai-audit/: 25 .draft files, with a table mapping each to its final file. Each fix is explained in its commit message. Two of the problems found:

Security: the image-scan step ran a moving third-party reference (ci-trivy-scan.yml.draft). The draft used aquasecurity/trivy-action@master in two separate steps. @master runs whatever is on that branch when the job starts, inside a job that holds the Docker socket and so controls every container on the runner: a change upstream would run in this pipeline without any change here. The two steps also meant that when the backend scan failed, the frontend scan never ran, so a red run hid half of the findings. The final step runs the pinned image aquasec/trivy:0.75.0 through docker run, scans both images in one loop, and fails only after both are scanned (|| status=1, then exit "\$status", because bash -e would otherwise stop at the first failure). The first run with it showed the findings of both images (screenshots 18–19).

Reliability: the health probe could hang on a frozen database (health.py.draft). The draft checked the database with SELECT 1 on a session from the request connection pool. It returned 503 when the database was stopped, so it looked correct in a quick test. Review asked what happens when the database accepts TCP but never answers. Reproduced with docker compose pause db, the probe waited 30 s on an already-open pooled connection until the gateway timed out, so the health check itself hung instead of reporting DOWN. The final version opens a fresh, unpooled connection with a 2 s connect timeout, under the container HEALTHCHECK's 2.5 s limit. Measured in the container, it answers 503 DOWN in 2.07 s and recovers without restarting the API.

What was learned. Both drafts passed the obvious test: the scan ran, and the probe said DOWN with the database stopped. The defects showed up only when asking what the code trusts and how it fails. For a CI step, that means who controls the code being executed. For a probe, it means what happens when a dependency is slow rather than gone. The other fixes recorded in ai-audit/ follow the same pattern.

5. Operations notes

Each SLO has one Prometheus alert and a runbook. The availability and login alerts fire at a 14.4× burn rate of the error budget (a 7-day budget gone in about 12 hours), with a traffic floor so a single failure on thin traffic cannot page; the latency alert is a plain threshold, p95 above 500 ms for 5 minutes. Each runbook is in in week-06-devops/docs/runbooks/: symptom, panels to check, likely causes, mitigation, follow-up. There is no Alertmanager, so alert state is read in Prometheus. Drill on 2 October: the database was stopped at 13:39:28 UTC under steady authenticated traffic; ApiHighErrorRate went pending at 13:39:49 and fired at 13:41:49 at an 85% 5xx ratio; the database was started at 13:43:43 and the alert resolved at 13:48:35 once the 5-minute window drained (screenshots 10–13).